CI/CD PIPELINE

From Source Code to Automated Deployment

Handbook

Applied AI Engineering

R Sumanth Simha

USN: 3BR23CD073

Git • GitHub • GitHub Actions • Pytest • Render

A practical learning resource covering the concepts, terminology, architecture and workflow behind Continuous Integration and Continuous Delivery/Deployment.

How to use this handbook

This handbook is designed to support a live workshop. Use it as a reference rather than reading every paragraph aloud. Introduce a real development problem, explain the concept that addresses it, show the workflow, and then demonstrate the implementation.

Suggested teaching principle: Explain why a practice exists before showing students how to configure it.

1. Introduction to Software Development

Software development is more than writing code. A reliable application passes through activities such as planning, design, implementation, testing, integration, deployment and maintenance.

Imagine five developers working on a student-management application. One handles authentication, another registration, another attendance, another fee management and another reports. Their work must eventually be combined into one application.

That creates practical questions:

• What if a developer introduces a bug?

• What if two developers change the same file?

• How can the team verify that combined changes still work?

• How can every change be tested consistently?

• How can a validated version be released reliably?

CI/CD practices help teams address these problems by making integration, validation and delivery more repeatable and automated.

2. Version Control: Git and GitHub

Version Control is the practice of recording changes to files over time. It helps developers inspect history, compare versions, restore earlier work and collaborate safely.

What is Git?

Git is a distributed version control system. Developers use it to track source-code changes, create branches, commit work and merge changes. Git can operate locally without GitHub.

What is GitHub?

GitHub is a cloud-based platform for hosting Git repositories and collaborating around them. It adds features such as remote repositories, Pull Requests, code review, Issues and GitHub Actions.

Git	GitHub
Version control system	Repository hosting and collaboration platform
Tracks changes in repositories	Hosts remote repositories and team workflows
Supports local commits, branches and merges	Provides Pull Requests, reviews and automation
Can be used without GitHub	Uses Git repositories

Remember: Git manages version history; GitHub provides a place and collaboration tools for Git repositories.

3. Software Development Lifecycle (SDLC)

The Software Development Lifecycle is a structured way to plan, build, test, release and maintain software. Common stages include:

1. Planning: Define the problem, users, scope and requirements.

2. Analysis: Examine requirements, constraints and feasibility.

3. Design: Define the architecture, data model and interfaces.

4. Development: Implement the application.

5. Testing: Check whether the application behaves as expected.

6. Deployment: Release the application into a target environment.

7. Maintenance: Monitor, fix and improve the application.

CI/CD supports and automates parts of development, integration, testing, release and deployment. It is not a replacement for the entire SDLC.

4. Continuous Integration (CI)

Continuous Integration is a software-development practice in which developers integrate changes into a shared codebase frequently. Automated checks are run on those changes so that problems can be detected early.

Why CI is useful

Suppose several developers work on separate features. Each feature may work on its author's laptop but fail when combined with the rest of the application. If integration happens infrequently, the source of the failure can be difficult to identify.

With CI, pushes or Pull Requests can trigger automated checks such as dependency installation, builds, tests, linting and other quality checks. Developers receive feedback soon after submitting changes.

Typical CI checks

• Install required dependencies.

• Build or validate the application.

• Run automated tests.

• Check formatting or code quality.

• Optionally run security or dependency checks.

Benefits: earlier defect detection, faster feedback, more consistent testing, fewer late integration surprises and improved collaboration.

Important: CI does not prove that software is bug-free. It only checks the conditions covered by the configured pipeline and tests.

5. Continuous Delivery

Continuous Delivery is a practice in which changes are automatically built, tested and prepared so that the application can be released reliably. The software is kept in a deployable state, while production deployment commonly involves a manual approval or release decision.

Example: a developer pushes a change; automated checks pass; the application is packaged and made ready for release; an authorized person decides when to deploy it to production.

6. Continuous Deployment

Continuous Deployment extends automation so that every change that passes the required pipeline checks can be deployed automatically to production, without a separate manual approval for each release.

Example: a developer fixes a bug, pushes the change, automated checks pass, and the updated application is deployed automatically.

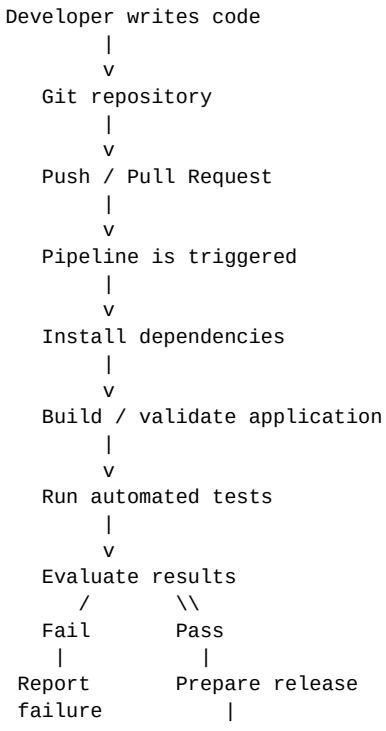
7. CI vs Continuous Delivery vs Continuous Deployment

Capability	CI	Continuous Delivery	Continuous Deployment
Frequent integration	Yes	Yes	Yes
Automated tests	Typical	Yes	Yes
Build and validation	Typical	Yes	Yes
Ready for release	Not necessarily	Yes	Yes
Production deployment	Not part of CI itself	Usually after approval	Automatic after required checks

Easy way to remember: CI asks whether changes pass the checks. Continuous Delivery keeps the application ready to release. Continuous Deployment releases qualifying changes automatically.

8. What is a CI/CD Pipeline?

A CI/CD pipeline is an automated sequence of tasks that takes source-code changes through validation, testing, building and, where configured, release and deployment.



```
    v
Deployment
    |
    v
Live application
```

A pipeline may contain source, build, test, release, deployment and monitoring stages. The exact stages and order depend on the application and team's delivery process.

9. GitHub Actions

GitHub Actions is GitHub's workflow-automation platform. It can run automated tasks in response to repository events, including tests, builds, scheduled jobs and deployments.

Workflow files are written in YAML and stored in the repository under `.github/workflows/`.

Core terminology

Term	Meaning
Workflow	An automated process defined in a YAML file.
Event / Trigger	An event that starts a workflow, such as push or pull_request.
Job	A group of steps executed on a runner.
Step	An individual task within a job.
Runner	The machine that executes a job, such as an Ubuntu runner.
Action	A reusable component that performs a task.
YAML	A human-readable configuration format used to define workflows.
Artifact	A file or package produced by a workflow and retained or shared as configured.

Common workflow triggers

```
on:
  push:

  pull_request:
```

The example configures a workflow to run for push and Pull Request events. In a real project, triggers can be restricted to particular branches or event types.

Simplified workflow example

```
name: Python CI

on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Run tests
        run: pytest
```

This workflow checks out the repository, configures Python, installs project dependencies and runs the test suite. A failed command normally causes the job to fail, which makes the result visible in GitHub Actions.

10. Automated Testing and Quality Gates

Automated testing means executing predefined test cases using software rather than manually checking every behavior each time.

Example: testing an addition function

```
# calculator.py
def add(a, b):
    return a + b

# test_calculator.py
from calculator import add

def test_addition():
    assert add(10, 20) == 30
```

If the function returns 30, the assertion passes. If the implementation is changed to return an incorrect value, the test fails.

A **quality gate** is a condition that must be satisfied before a pipeline proceeds. For example, a deployment job can depend on a test job succeeding. A test suite is only as useful as the behaviors it actually covers.

11. Deployment and Hosting

Deployment is the process of releasing an application into a target environment so that it can be used. A program running on a developer's laptop is not automatically accessible to other people; it needs to run in an environment reachable by its intended users.

What is Render?

Render is a cloud platform that can host web applications, including Python web applications. A connected Git repository can be used as the source for building and deploying a service.

Typical deployment flow

1. A developer pushes code to the connected GitHub branch.

2. The hosting platform detects the update, according to its deployment configuration.

3. The platform obtains the source code and runs the configured build process.

4. The application is started using its configured start command.

5. The deployed service becomes available through its public URL.

Deployment behavior depends on the hosting service's settings. For example, deployment may occur on every commit, after CI checks pass, or only when manually requested.

12. CI Checks and Deployment Gating

A deployment gate connects successful validation to the decision to deploy. In a gated workflow, the deployment stage should proceed only when the required CI checks have passed.

For a Render service configured to deploy after CI checks pass, the relevant GitHub checks must be associated with the commit Render is evaluating. A workflow running elsewhere, on another branch or against another commit may not satisfy that requirement.

Teaching tip: Show students the GitHub commit's Checks section, then compare it with the Render service's Events and deployment logs. This teaches them to trace a pipeline across systems rather than assuming that one green screen means every stage ran.

13. Secrets and Environment Variables

Applications often need sensitive configuration such as API keys, database credentials and authentication secrets. These values should not be hardcoded into source code or committed to a public repository.

• Use GitHub Actions secrets for values needed by workflows.

• Use the hosting platform's environment-variable settings for runtime configuration.

• Limit access to secrets and rotate them if they are exposed.

• Never publish deployment-hook URLs, passwords, tokens or private credentials.

```
# Example GitHub Actions expression
${{ secrets.SECRET_NAME }}
```

14. Deployment Environments

Environment	Purpose
Development	Used by developers while implementing and checking changes.
Testing / Staging	Used to validate changes before production release.
Production	The environment used by actual customers or end users.

Separating environments helps teams validate changes before releasing them to users. Not every small project needs all three environments, but students should understand why larger systems often use them.

15. CI/CD in Collaborative Development

A typical team workflow combines Git branching, Pull Requests, automated checks and deployment:

1. A developer creates a feature branch.

2. The developer implements a change and commits it.

3. The developer pushes the branch to GitHub.

4. A Pull Request is opened.

5. GitHub Actions runs the configured checks.

6. The team reviews the changes and checks the results.

7. The Pull Request is merged when review and repository rules are satisfied.

8. The deployment process runs according to the configured release policy.

Branch protection rules can require successful status checks before a Pull Request is merged. This is a repository policy; a workflow failing does not automatically block every merge unless the repository is configured to enforce it.

16. Benefits and Limitations of CI/CD

Benefits

• Faster feedback on code changes.

• Less repetitive manual testing and deployment work.

• Earlier detection of defects.

• More consistent release processes.

• Better visibility into failures through logs and status checks.

• Improved collaboration across development teams.

Challenges

• Poorly designed tests can miss important defects.

• Flaky tests can produce inconsistent results.

• Incorrect workflow configuration can stop a pipeline.

• Secrets and permissions must be managed carefully.

• Builds and deployments can fail because of dependency or environment differences.

• Monitoring and rollback strategies are still needed.

CI/CD improves repeatability and speed; it does not remove the need for engineering judgment, good tests, security practices or operational monitoring.

17. Workshop Implementation Project

The workshop will use a small Student Expense Tracker so students can focus on the delivery workflow rather than a complicated application.

Technology	Responsibility
Python	Application logic

Technology	Responsibility
Flask	Web framework
HTML / CSS	User interface
Pytest	Automated testing
Git	Version control
GitHub	Repository hosting and collaboration
GitHub Actions	Workflow automation and CI
Render	Application hosting and deployment

Implementation phases

1. Build: Create and run the application locally.

2. Version control: Create a repository and practise commits, branches and Pull Requests.

3. Testing: Write tests for core application behavior.

4. CI: Configure GitHub Actions to run tests on pushes and Pull Requests.

5. CD: Configure deployment to occur after the required CI checks pass.

6. Failure demonstration: Introduce a deliberate bug, observe the failed test, fix it and verify the successful pipeline.

18. Suggested Live Demonstration

Use one small change throughout the demonstration so students can follow the complete lifecycle.

1. Show the working application locally and its deployed URL.

2. Create a feature branch and make a small change.

3. Commit and push the change.

4. Open a Pull Request and show the automated CI check.

5. Introduce an intentional bug and observe a failed test.

6. Fix the bug and push the correction.

7. Show the successful check and the deployment gate.

8. Inspect the hosting platform's deployment events and logs.

9. Open the live application and verify the change.

If the project deploys directly on every commit, configure the deployment policy before demonstrating a CI-gated release. Otherwise, a broken commit may deploy before the tests finish.

19. Quick Recap

The following tools and practices work together in a modern software delivery workflow:

Tool / Concept	Primary responsibility
Git	Tracks source-code versions and changes.
GitHub	Hosts repositories and supports collaboration.
GitHub Actions	Automates workflows such as testing and deployment.
Pytest	Runs automated Python tests.
Render	Hosts and deploys the web application.
CI/CD	Connects integration, validation and software delivery practices.

Final takeaway: Modern software development is not limited to writing code. Developers also need to manage versions, collaborate, test changes, automate repetitive work and release applications reliably. CI/CD connects these activities into a repeatable software-delivery process.